智维Agent · SmartOpsAgent

面向中小企业的可信 AI 智能运维与知识协同平台

技术文档

项目类型：AI Agent + RAG + 智能运维 + 知识协同 + 安全自动化

版本：v0.1.0（竞赛交付版）

日期：2026-10-24

1. 项目概述

智维Agent 面向中小企业运维场景，将大模型能力约束在"证据驱动、风险可控、全程留痕"的闭环内，解决通用问答机器人不可信、不可控、难落地的问题。系统覆盖知识获取 → 知识检索 → 证据验证 → AI 分析 → Agent 规划 → 风险控制 → 安全执行 → 结果验证 → 审计留痕的完整链路。

AI Agent | RAG 证据验证 | SSE 流式 | 思考过程可视化 | 多模态 | 风险策略引擎 | 审计留痕

2. 系统架构

层	技术栈	职责
前端	Vue 3 + Vite + Element Plus + Pinia + ECharts + Three.js（共享逻辑层 TypeScript）	运维控制台、AI 对话面板（语音/图像证据/流式思考可视化）、3D 与图表可视化
后端	Spring Boot（:8080）	运维数据、账户与权限、附件会话存储、审计
AI 服务	FastAPI（:8001）	LLM 网关（可切换 Provider）、对话与 SSE 流式接口、RAG 编排
Agent 框架	Python	planner（规划）/ executor（执行）/ inspector（核查）/ memory（记忆）/ policies（策略）

用户提问（文本 / 语音 / 图像证据） → 前端对话面板（Vite /ai 代理） → ai-service：RAG 检索 → 证据验证 → Prompt 组装（系统提示 + 知识证据 + 图像证据） → LLM 网关（mock / Ollama / OpenAI 兼容，可切换 Provider） → SSE 回流：thinking（思考阶段）→ token（流式正文）→ done（引用 + 可信度评分） → 高风险操作：策略引擎评估 → 人工审批 → 执行 → 审计留痕

3. 核心技术创新

3.1 可信 AI 闭环

Agent 不直接执行高风险操作：planner 拆解故障为可执行步骤，policies 策略引擎评估操作风险，高风险动作必须经人工审批；inspector 核查执行结果，全程审计留痕，体现"可信 AI"设计理念。

3.2 RAG 证据验证

混合检索（关键词 + 语义）结合证据验证机制，知识条目带版本与来源引用；回答附 citations 与 trust_score，显著降低错误版本知识干扰，回答可追溯、可核对。

3.3 AI 思考过程可视化 + SSE 流式输出

ai-service 的 `chat_stream` 按"图像证据解析 → 知识检索 → 证据验证 → Agent 规划 → 风险判定 → 生成回答"阶段产出 thinking 事件，前端逐步渲染思考过程并叠加流式 token 光标，将"黑盒回答"变

为"透明推理"。

3.4 多模态交互

- **语音输入**：基于 Web Speech API 的渐进增强实现（Chrome/Edge 可用，不支持时自动隐藏入口）；识别文本仅填入输入框，不自动发送、不上传音频。
- **图像证据**：截屏/拍照/图片附件本地暂存，随消息仅将文件名传入 ai-service 参与模拟解析（思考链路出现"图像证据解析"阶段）；图片内容不上传。文本/日志附件走私有附件会话，仅保存证据、不参与模型分析，形成清晰的隐私边界。
- **可视化**：Three.js 3D 拓扑与 ECharts 图表呈现监控与知识分析结果。

4. 安全与可信设计

- 权限/风险策略引擎：操作分级，高危命令默认拒绝或转人工审批。
- 人工审批与审计留痕：关键动作全程可追溯。
- 附件隐私设计：普通会话仅传图像文件名；私有附件会话内容不参与模型分析；提示用户遮盖密码、密钥等敏感信息。
- 管理端安全：管理员 MFA、注册安全策略、会话管理。

5. 工程化与质量

- **TypeScript 部分改造**：共享逻辑层（api / domain / utils / types，48 个模块）迁移至 TS，组件与 store 保持 JS；`npm run typecheck`（tsc --noEmit）纳入质量门禁。
- **构建分包**：manualChunks 收敛至共享配置；zrender 从 echarts 拆出（vendor-echarts 510 kB → 346 kB），Three.js 保持懒加载。
- **测试**：Node test runner 共 458 项单元/契约测试全部通过；另附 Playwright 浏览器级附件链路用例。
- **仓库卫生**：本地缓存与运行时数据移出跟踪，跟踪文件 3091 → 1585；README / CHANGELOG / LICENSE / .env.example 齐备。

```
cd frontend
npm test             # 458 项测试
npm run typecheck    # tsc --noEmit
npm run build        # 用户层构建
npm run build:admin # 管理员层构建
```

6. 运行与演示

```
# 后端（:8080）
cd backend && ./mvnw spring-boot:run
# AI 服务（:8001）
cd ai-service && uvicorn app.main:app --port 8001
# 前端（开发，/ai 代理至 8001）
cd frontend && npm run dev
```

演示动线：进入 Agent 对话 → 语音描述故障（如"MySQL 连接数打满"）→ 截取屏幕作为图像证据 → 观察思考过程六阶段与流式回答 → 查看引用证据与可信度评分 → 切换附件会话演示隐私边界。

7. 路线图

- 流式多模态增强：图像内容级理解（OCR / 多模态大模型）在隐私授权后接入。
- 端侧推理：Transformers.js / WebGPU 本地小模型，进一步降低数据出域。
- 多节点会话与 MFA 强化、团队协作空间。